Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 3
по дисциплине «Типы и структуры данных»

Тема Длинная арифметика

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Фибоначчиевая система счисления 5

2 Конструкторская часть **6**

2.1 Структура класса BigIntFibonacci 6

2.2 Операторы сравнения 7

2.3 Арифметические операции 8

3 Технологическая часть **12**

3.1 Реализация и выбор средств 12

3.2 Тестирование программы 12

ЗАКЛЮЧЕНИЕ **14**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **15**

ПРИЛОЖЕНИЕ А **16**

ВВЕДЕНИЕ

В современных условиях разработки программных алгоритмов зачастую возникает необходимость обработки чисел, размеры которых значительно превосходят возможности стандартных типов данных, предоставляемых языками программирования. В связи с этим возникает потребность в использовании специальных структур данных и методов, позволяющих выполнять арифметические операции с числами произвольной длины. Решением данной задачи является длинная арифметика — комплекс технологий и алгоритмов, позволяющих эффективно работать с большими числами, обеспечивая точность и корректность вычислений вне пределов стандартных типов данных.

Целью работы является реализация алгоритмов длинной арифметики для работы с числами в фибоначчиевой системе счисления. Для достижения поставленной задачи необходимо выполнить следующие пункты:

1) ознакомиться с теоретическими основами представления целых чисел в системе Фибоначчи и особенностями их записи;

2) реализовать структуру BigIntFibonacci, обеспечивающую хранение чисел в фибоначчиевой системе счисления и выполнение основных арифметических операций, таких как сложение, вычитание, умножение, нахождение остатка от деления;

3) провести тестирование реализованных методов на различных примерах для проверки их корректности.

1 Аналитическая часть

Длинная арифметика — это совокупность методов и структур данных, позволяющих выполнять арифметические операции с числами (сравнение, сложение, умножение, деление), превышающими стандартные размеры встроенных типов данных. Операции выполняются в ней поразрядно [1]. Необходимость в длинной арифметике возникает в случае, когда требуется проводить арифметические операции или хранить большие числа, которые не помещаются в стандартные типы данных

1.1 Фибоначчиевая система счисления

Фибоначчиева система счисления — это смешанная система счисления для целых чисел, основанная на числах Фибоначчи: $F_2 = 1, \quad F_3 = 2, \quad F_4 = 3, \quad F_5 = 5, \quad F_6 = 8, \quad$ и так далее. В ее основе лежит теорема Цекендорфа [1] — любое неотрицательное целое число единственным образом представимо в виде суммы некоторого набора попарно различных чисел Фибоначчи с индексами, большими единицы, не содержащего пар соседних чисел Фибоначчи. Таким образом, неотрицательное целое число a можно единственным образом представить последовательностью битов $\ldots \varepsilon_k \ldots \varepsilon_4 \varepsilon_3 \varepsilon_2$, где $\varepsilon_k \in \{0, 1\}$, так, что

$$a = \sum_k \varepsilon_k F_k, \tag{1.1}$$

при этом последовательность $\{\varepsilon_k\}$ содержит лишь конечное число единиц и не имеет пар соседних единиц:

$$\forall k \geq 2\colon (\varepsilon_k = 1) \Rightarrow (\varepsilon_{k+1} = 0). \tag{1.2}$$

За исключением последнего свойства, данное представление аналогично двоичной системе счисления. Также из теоремы Цекендорфа следует важное ограничение: любые два соседних числа Фибоначчи не могут одновременно входить в разложение, поэтому комбинация 11 запрещена в фибоначчиевой системе счисления.

Вывод

Были приведены все необходимые определения, формулы и ограничения для дальнейшей реализации.

2 Конструкторская часть

2.1 Структура класса BigIntFibonacci

Основные поля класса

Класс `BigIntFibonacci` содержит одно поле — `vector<int> digits` — объект типа `vector`, содержащий цифры разложения числа в факториальной системе счисления, где младший разряд находится в начале.

Конструкторы класса

`BigIntFibonacci(const string& s)` — инициализация из строки чисел, разбивая строку на цифры в обратном порядке.

`BigIntFibonacci(const vector<int>& v)` — инициализация из вектора цифр, с вызовом нормализации.

`BigIntFibonacci()` — создание объекта с числом 0.

`BigIntFibonacci(const BigIntFibonacci& other)` — копирование числа из другого объекта.

Методы класса

`void normalize()` — метод для нормализации представления числа в фибоначчиевой системе счисления. Он преобразует произвольное числовое представление в каноническое, исключая подряд идущие единицы.

Пусть имеется вектор целых чисел:

$$\mathbf{a} = [a_0, a_1, \ldots, a_{n-1}],$$

где $a_i \in \mathbb{Z}$.

— для каждого $i = 0, \ldots, n-2$:

$$\begin{cases} r_i = a_i \bmod 2, \\ a_i \leftarrow \left[\frac{a_i}{2}\right], \\ a_{i+1} \leftarrow a_{i+1} + r_i. \end{cases} \tag{2.1}$$

— если найдены подряд идущие единицы, то:

$$\text{при } a_i = 1,\ a_{i+1} = 1: \quad \begin{cases} a_i \leftarrow 0, \\ a_{i+1} \leftarrow 0, \\ a_{i+2} \leftarrow a_{i+2} + 1. \end{cases} \tag{2.2}$$

при условии, что $i + 2 < n$.

Процесс повторяется для всего вектора, пока не достигнется каноническая форма, при которой все элементы будут равны 0 или 1, и не будет последовательных единиц.

2.2 Операторы сравнения

В силу выбранного варианта хранения числа в виде вектора `digits` 2.1, операторы сравнения реализуются при помощи последовательных сравнений элементов этого вектора. Алгоритмы производят поэлементное сравнение с учетом длины векторов и правил сравнения чисел в фибоначчиевой системе.

Реализованы перегрузки 6 операторов сравнения.

1) `operator==` — проверка чисел в фибоначчиевой системе на равенство. Если векторы `digits` двух чисел совпадают по длине и по соответствующим элементам, то вернуть **истину**, иначе — **ложь**.

2) `operator!=` — проверка чисел в фибоначчиевой системе на неравенство. Если результаты оператора равенства отрицательны, то вернуть **истину**, иначе — **ложь**.

3) `operator<` — оператор "строго меньше".

— определить максимальную длину чисел:

$$n = \max(\text{digits.size(), other.digits.size()}) \tag{2.3}$$

— выполнить поэлементное сравнение с высоты $i = n - 1$ до 0:

— для каждого i определить цифры:

$$a_i = \begin{cases} \text{digits}[i], & \text{если } i < \text{digits.size()} \\ 0, & \text{иначе} \end{cases} \tag{2.4}$$

$$b_i = \begin{cases} \text{other.digits}[i], & \text{если } i < \text{other.digits.size()} \\ 0, & \text{иначе} \end{cases} \tag{2.5}$$

— сравнить цифры:

Если a_i меньше b_i, то вернуть **истину**. Если a_i больше b_i, то вернуть **ложь**.

— если все сравнения прошли без вывода (то есть все $a_i = b_i$), то числа равны, и

результат — **ложь**:

$$\text{Если} \quad \forall i, \quad a_i = b_i, \quad \text{то} \quad \text{вернуть} \quad \textbf{ложь} \tag{2.6}$$

4) `operator<=` — оператор "меньше или равно".
— если $*this <$ other или $*this ==$ other, то вернуть **истину**;
— иначе вернуть **ложь**.

5) `operator>` — оператор "строго больше".
— если $!(*this <=$ other$)$, то вернуть **истину**;
— иначе вернуть **ложь**.

6) `operator>=` — оператор "больше или равно".
— если $!(*this <$ other$)$, то вернуть **истину**;
— иначе вернуть **ложь**.

2.3 Арифметические операции

Алгоритм сложения в системе Фибоначчи

Ниже описан алгоритм сложения чисел в фибоначчиевой системе счисления:

1) инициализация результата: создается вектор `result` длиной `max(digits.size(), other.digits.size()) + 1`, заполненный нулями;

2) цикл по индексам `i` от 0 до `maxSize - 1`, где `maxSize` — максимальная длина исходных чисел). Для каждого `i` берутся разряды исходных чисел `a_digit` и `b_digit`:

$$a_digit = \begin{cases} digits[i], & \text{если } i < digits.size() \\ 0, & \text{иначе} \end{cases} \tag{2.7}$$

$$b_digit = \begin{cases} other.digits[i], & \text{если } i < other.digits.size() \\ 0, & \text{иначе} \end{cases} \tag{2.8}$$

3) значения `a_digit` и `b_digit` складываются и добавляются к `result[i]`:

$$result[i] = result[i] + a_digit + b_digit \tag{2.9}$$

После заполнения `result`, вызывается метод `normalize()` 2.1, который приводит число к каноническому виду по правилам системы Фибоначчи. Если `digits[i] > 1`, то делим значение на 2, оставляя остаток в `digits[i]`, и переносим целую часть `carry` в следующий разряд `i+1`:

$$\begin{cases} carry = \left[\frac{digits[i]}{2}\right], \\ digits[i] = digits[i] \bmod 2, \\ digits[i+1] = digits[i+1] + carry. \end{cases} \tag{2.10}$$

Если в соседних разрядах обнаружены 1 и 1, то они заменяются на 0 и 0, а в следующий разряд добавляется 1 (по правилам уникального разложения в системе Фибоначчи):

$$\begin{cases} digits[i] = 0, \\ digits[i+1] = 0, \\ digits[i+2] = digits[i+2] + 1. \end{cases} \tag{2.11}$$

Также требуется удалить ведущие нули: пока `digits.back() = 0` и `digits.size() > 1`, удаляем последний элемент.

Итоговый результат — это число, приведенное к каноническому виду, возвращается как объект `BigIntFibonacci` 2.1.

Алгоритм вычитания в системе Фибоначчи

Ниже описан алгоритм вычитания чисел в фибоначчиевой системе счисления:

1) проверка условий: если первое число `a` меньше второго `b` (`a < b`), то для простоты можно вернуть нулевое число или сигнализировать об отрицательном результате (в текущем коде возвращается ноль);

2) инициализация результата — копия вектора `digits` числа `a`;

3) вычитание соответствующих разрядов. Для каждого разряда $i \in [\,0,$ other.digits.size()):

$$result[i] = result[i] - other.digits[i] \tag{2.12}$$

После поэлементного вычитания вызывается метод `normalize()` 2.1, который корректирует число по правилам системы Фибоначчи. В случае, если в разряде получился отрицательный результат, необходимо занять из следующего разряда:

$$\begin{cases} result[i] = result[i] + 2, \\ result[i+1] = result[i+1] - 1, \end{cases} \quad \text{если } result[i] < 0 \tag{2.13}$$

Если обнаружены последовательные единицы, то:

$$\begin{cases} result[i] = 0, \\ result[i+1] = 0, \\ result[i+2] = result[i+2] + 1, \end{cases} \quad \text{если } result[i] == 1 \text{ и } result[i+1] == 1 \tag{2.14}$$

Также требуется удалить ведущие нули: пока `digits.back() = 0` и `digits.size() > 1`, удаляем последний элемент.

Итоговое число `result` после нормализации является результатом вычитания. Возвращается объект `BigIntFibonacci` 2.1 с полученными разрядами.

Алгоритм умножения в системе Фибоначчи

Ниже описан алгоритм умножения чисел в фибоначчиевой системе счисления:

1) инициализация нулями результирующего вектора `result` размером `digits.size() + other.digits.size()`:

$$result.resize(digits.size() + other.digits.size(),\ 0) \tag{2.15}$$

.

2) двойной цикл перебирает разряды множимых. Для каждого $i \in [\,0,$ other.digits.size()) и для каждого $j \in [\,0,$ other.digits.size()):

$$result[i+j] = result[i+j] + digits[i] \cdot other.digits[j] \tag{2.16}$$

3) после заполнения `result` вызывается метод `normalize()` 2.1, который приводит число к каноническому виду по правилам системы Фибоначчи;

4) итоговое число `result` после нормализации является произведением двух чисел в системе Фибоначчи;

5) создается объект `FibNumber` 2.1 из полученного вектора `result` и возвращается.

Алгоритм нахождения остатка от деления

Ниже описан алгоритм нахождения остатка от деления в фибоначчиевой системе счисления:

1) инициализация переменной `remainder` значением 0: `BigIntFibonacci(0)`;

2) обход разрядов делимого `a` в обратном порядке, начиная со старшего: на каждом шаге сдвигаем `remainder` влево на один разряд (эквивалентно умножению на `F`, где `F` — число Фибоначчи). В `remainder` добавляем текущий разряд делимого `a.digits[i]` (0 или 1):

$$\text{remainder}[i] = 2 \cdot \text{remainder}[i+1] + a.\text{digits}[i], \quad i = n-1, n-2, \ldots, 0 \tag{2.17}$$

3) после этого для поддержания `remainder` в каноническом виде необходимо выполнить процедуру редукции: пока `remainder` $\geq$ `b`, вычитаем `b` из `remainder`, используя алгоритм вычитания, описанный выше в пункте 2.3;

4) после завершения цикла по всем разрядам, `remainder` содержит остаток от деления `a` на `b`;

5) возвращается объект `BigIntFibonacci` с разрядами `remainder`.

Вывод

Был разработан класс BigIntFibonacci для работы с числами в фибоначчиевой системе счисления.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации использовалась среда разработки $Visual\ Studio\ Code$ [2]. Язык программирования, на котором была выполнена реализации алгоритмов — $C++$. Реализация программы представляет собой класс `BigIntFibonacci` для работы с числами в фибоначчиевой системе счисления. Класс содержит все методы и перегрузки стандартных операторов языка $C++$, реализации которых были описаны в предыдущем разделе. В листинге А.1 представлена программная реализация описанного класса.

3.2 Тестирование программы

Ниже в таблице 3.1 приведено тестирование программы и её проверка на ожидаемый/получаемый результат.

Таблица 3.1 — Результаты тестирования

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на ввод нечисловых значений	A B	Некорректный ввод. Попробуйте снова.	Некорректный ввод. Попробуйте снова.
Проверка на ввод чисел не в фибоначчиевой системе счисления	4 5	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.
Проверка на ввод чисел с соседними единицами	11 101	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.

Таблица 3.1 — Продолжение

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на ввод числа b, большего чем число a	100101 101	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: Некорректный ввод. Попробуйте снова.
Проверка корректность работы программы	101 10	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: a > b a - b = 100 a + b = 1000 a * b = 101 a % 3 = 1	Введите число a в фибоначчиевой системе счисления: Введите число b в фибоначчиевой системе счисления: a > b a - b = 100 a + b = 1000 a * b = 101 a % 3 = 1

Вывод

Было проведено тестирование программы на работоспсобность.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы: реализация алгоритмов длинной арифметики для работы с числами в фибоначчиевой системе счисления.

Также были выполнены все задачи:

1) изучены теоретические основы представления целых чисел в системе Фибоначчи и особенности их записи. Были рассмотрены свойства фибоначчиевых кодов и правила нормализации чисел в данной системе;

2) реализован класс `BigIntFibonacci`, обеспечивающий хранение чисел в фибоначчиевой системе и выполнение основных арифметических операций: сложения, вычитания, умножения и нахождения остатка от деления;

3) проведено тестирование реализованных методов на различных примерах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.

2. Visual Studio Code documentation. [Электронный ресурс]. URL: `https://code.visualstudio.com/docs` (дата обращения: 22 октября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <cctype>

using namespace std;

class BigIntFibonacci {
  public:

  vector<int> digits;

  BigIntFibonacci(const string& s) {
    for (int i = s.size() - 1; i >= 0; i--)
    digits.push_back(s[i] - '0');
    normalize();
  }

  BigIntFibonacci(const vector<int>& v) : digits(v) {
    normalize();
  }

  BigIntFibonacci() {
    digits = {0};
  }

  BigIntFibonacci(const BigIntFibonacci& other) {
    digits = other.digits;
  }

  bool operator==(const BigIntFibonacci& other) const {
    return digits == other.digits;
  }

  bool operator!=(const BigIntFibonacci& other) const {
    return !(*this == other);
  }

  bool operator<(const BigIntFibonacci& other) const {
    int n = max(digits.size(), other.digits.size());
    for (int i = n - 1; i >= 0; i--) {

      int a_digit, b_digit;
```

```

      if (i < digits.size())
      a_digit = digits[i];
      else
      a_digit = 0;

      if (i < other.digits.size())
      b_digit = other.digits[i];
      else
      b_digit = 0;

      if (a_digit < b_digit)
      return true;
      if (a_digit > b_digit)
      return false;

    }
    return false;
  }

  bool operator<=(const BigIntFibonacci& other) const {
    return (*this < other) || (*this == other);
  }

  bool operator>(const BigIntFibonacci& other) const {
    return !(*this <= other);
  }

  bool operator>=(const BigIntFibonacci& other) const {
    return !(*this < other);
  }

  void normalize() {
    bool changed;
    do {
      changed = false;
      for (int i = 0; i < digits.size(); i++) {
        if (digits[i] > 1) {
          int carry = digits[i] / 2;
          digits[i] %= 2;
          if (i + 1 >= digits.size()) digits.push_back(0);
          digits[i + 1] += carry;
          changed = true;
        }
      }

      for (int i = 0; i + 1 < digits.size(); i++) {
```

```
        if (digits[i] == 1 && digits[i + 1] == 1) {
          digits[i] = 0;
          digits[i + 1] = 0;
          if (i + 2 >= digits.size()) digits.push_back(0);
          digits[i + 2] += 1;
          changed = true;
        }
      }
      while (digits.size() > 1 && digits.back() == 0)
      digits.pop_back();

    } while (changed);
  }

  BigIntFibonacci operator+(const BigIntFibonacci& other) const {
    vector<int> result;
    int maxSize = max(digits.size(), other.digits.size());
    result.resize(maxSize + 1, 0);

    for (int i = 0; i < maxSize; i++) {
      int a_digit = (i < digits.size()) ? digits[i] : 0;
      int b_digit = (i < other.digits.size()) ? other.digits[i] : 0;
      result[i] += a_digit + b_digit;
    }
    BigIntFibonacci res(result);
    res.normalize();
    return res;
  }

  BigIntFibonacci operator-(const BigIntFibonacci& other) const {

    if (*this < other)
    return BigIntFibonacci("0");
    vector<int> result = digits;

    for (int i = 0; i < other.digits.size(); i++)
    result[i] -= other.digits[i];

    BigIntFibonacci res(result);
    res.normalize();
    return res;
  }

  BigIntFibonacci operator*(const BigIntFibonacci& other) const {
    vector<int> result(digits.size() + other.digits.size(), 0);
    for (size_t i = 0; i < digits.size(); i++)
    for (size_t j = 0; j < other.digits.size(); j++)
```

```
    result[i + j] += digits[i] * other.digits[j];
    BigIntFibonacci res(result);
    res.normalize();
    return res;
  }

  BigIntFibonacci operator%(int divisor) const {

    if (divisor <= 0) return BigIntFibonacci("0");

    BigIntFibonacci remainder("0");
    for (int i = (int)digits.size() - 1; i >= 0; i--) {

      remainder.digits.insert(remainder.digits.begin(), 0);
      if (remainder.digits.size() == 0)
      remainder.digits = {0};
      remainder.digits[0] = digits[i];
      remainder.normalize();

      int count = 0;
      while (remainder >= BigIntFibonacci(to_string(divisor))) {

        remainder = remainder - BigIntFibonacci(to_string(divisor));
        ++count;

      }

    }
    return remainder;
  }

};

ostream& operator<<(ostream& output, const BigIntFibonacci& num) {
  for (int i = num.digits.size() - 1; i >= 0; i--)
  output << num.digits[i];
  return output;
}

istream& operator>>(istream& input, BigIntFibonacci& num) {
  string s;
  input >> s;
  num.digits.clear();

  for (int i = s.size() - 1; i >= 0; i--)
  num.digits.push_back(s[i] - '0');

```

```
  num.normalize();
  return input;
}

BigIntFibonacci stringToFibNumber(const string& s) {
  return BigIntFibonacci(s);
}

bool isValidNumberString(const string& s) {
  if (s.empty()) return false;
  for (char c : s)
  if (c != '0' && c != '1')
  return false;

  for (int i = 0; i + 1 < s.size(); i++)
  if (s[i] == '1' && s[i + 1] == '1')
  return false;

  return true;
}

bool isValidFibonacciNumber(const string& s) {
  if (!isValidNumberString(s))
  return false;

  try {
    BigIntFibonacci num(s);
    return true;
  } catch (...) {
    return false;
  }
}

int main() {

  string a1, b1;
  do {
    cout << "Введите число a в фибоначчиевой системе: ";
    cin >> a1;
    cout << "Введите число b в фибоначчиевой системе: ";
    cin >> b1;

    bool validA = isValidFibonacciNumber(a1);
    bool validB = isValidFibonacciNumber(b1);

    if (!validA || !validB) {
      cout << "Некорректный ввод. Попробуйте снова." << endl;
```

```
233        continue;
234      }
235
236      BigIntFibonacci a = a1;
237      BigIntFibonacci b = b1;
238
239      if (!(a > b)) {
240        cout << "Число a должно быть больше числа b. Попробуйте снова." << endl;
241      } else {
242        break;
243      }
244    } while (true);
245    BigIntFibonacci a = a1, b = b1;
246
247    if (a > b)
248    cout << "a > b" << endl;
249    else if (a < b)
250    cout << "a < b" << endl;
251    else
252    cout << "a == b" << endl;
253
254    cout << "a - b = " << a - b << endl;
255
256    cout << "a + b = " << a + b << endl;
257
258    cout << "a * b = " << a * b << endl;
259
260    cout << "a % 3 = " << a % 3 << endl;
261
262    return 0;
263  }
```

Листинг А.1 — Программная реализация разных алгоритмов умножения матриц